

Number Theoretic Algorithms (5)

Recall Basic Number theoretic concepts

Solving Modular Linear Equations

Euclid's and Extended Euclid's Algorithms for solving Modular Linear Equations.

```
EXTENDED-EUCLID (a, b)
{  if b = 0
    return(a,1,0)    // d,x,y
  else (d', x', y') = EXTENDED-EUCLID (b, a mod b)
    (d, x, y) = (d', y', x' - ⌊a/b⌋y')
    return (d, x, y)
}
```

$$T(n) = O(\log(\min(a, b)))$$

a	b	$\lfloor a/b \rfloor$	d	x	y
99	78	1	3	-11	14
78	21	3	3	3	-11
21	15	1	3	-2	3
15	6	2	3	1	-2
6	3	2	3	0	1
3	0	—	3	1	0

Algorithm for Solving modular linear equations

Finding solutions to the equation $ax \cong b(\text{mod } n)$;

MODULAR-LINEAR-EQUATION-SOLVER(a, b, n)

1 $(d, x', y') = \text{EXTENDED-EUCLID}(a, n)$

2 **if** $d \mid b$

3 $x_0 = x'(b/d) \text{ mod } n$

4 **for** $i = 0$ **to** $d - 1$

5 print $(x_0 + i(n/d)) \text{ mod } n$

6 **else** print “no solutions”

$T(n) = O(d + \text{Log}(\min(a, n)))$

For $14x \cong 30 (\text{mod } 100)$ (here, $a = 14$, $b = 30$, and $n = 100$).

Calling EXTENDED-EUCLID we can obtain $(d, x', y') = (2, -7, 1)$

Since $d \mid b$ there exists two solutions x_0 and x_1 95 and 45 respectively.

Algorithm for Solving Modular Exponentiation

MODULAR-EXPONENTIATION(a, b, n)

```

1   $c = 0$ 
2   $d = 1$ 
3  let  $\langle b_k, b_{k-1}, \dots, b_0 \rangle$  be the binary representation of  $b$ 
4  for  $i = k$  downto 0
5       $c = 2c$ 
6       $d = (d \cdot d) \bmod n$ 
7      if  $b_i == 1$ 
8           $c = c + 1$ 
9           $d = (d \cdot a) \bmod n$ 
10 return  $d$ 

```

If b represented by n bits then

$T(n) = O(n)/O(\log n)$

Any integer n can be
represented by $\log n$ bits !

i	9	8	7	6	5	4	3	2	1	0
b_i	1	0	0	0	1	1	0	0	0	0
c	1	2	4	8	17	35	70	140	280	560
d	7	49	157	526	160	241	298	166	67	1

The results of MODULAR-EXPONENTIATION when computing $a^b \bmod n$, where $a = 7$, $b = 560 = [1000110000]$, and $n = 561$. The values are shown after each execution of the for loop. The final result is 1.

Miller-Rabin Randomized Primality Test and Analysis

MILLER-RABIN(n, s)

```
1  for  $j = 1$  to  $s$ 
2       $a = \text{RANDOM}(1, n - 1)$ 
3      if WITNESS( $a, n$ )
4          return COMPOSITE           // definitely
5  return PRIME                       // almost surely
```

WITNESS(a, n)

```
1  let  $t$  and  $u$  be such that  $t \geq 1$ ,  $u$  is odd, and  $n - 1 = 2^t u$ 
2   $x_0 = \text{MODULAR-EXPONENTIATION}(a, u, n)$ 
3  for  $i = 1$  to  $t$ 
4       $x_i = x_{i-1}^2 \bmod n$ 
5      if  $x_i == 1$  and  $x_{i-1} \neq 1$  and  $x_{i-1} \neq n - 1$ 
6          return TRUE
7  if  $x_t \neq 1$ 
8      return TRUE
9  return FALSE
```

Loop in main function runs S times
Witness function $n-1 = 2^t \cdot u$ takes $\log n$ times
Modular exponentiation takes $\log n$ time
For loop runs t times $\cong n$
so
 $T(n) = S \cdot (\log n + \log n + n)$ integer step.
Which is very fast.

An example of the application of the Chinese remainder theorem, suppose we are given the two equations

$$a \equiv 2 \pmod{5},$$

$$a \equiv 3 \pmod{13},$$

so that $a_1 = 2$, $n_1 = m_2 = 5$, $a_2 = 3$, and $n_2 = m_1 = 13$, and we wish to compute $a \bmod 65$, since $n = n_1 n_2 = 65$. Because $13^{-1} \equiv 2 \pmod{5}$ and $5^{-1} \equiv 8 \pmod{13}$, we have

$$c_1 = 13(2 \bmod 5) = 26,$$

$$c_2 = 5(8 \bmod 13) = 40,$$

and

$$\begin{aligned} a &\equiv 2 \cdot 26 + 3 \cdot 40 \pmod{65} \\ &\equiv 52 + 120 \pmod{65} \\ &\equiv 42 \pmod{65}. \end{aligned}$$

	0	1	2	3	4	5	6	7	8	9	10	11	12
0	0	40	15	55	30	5	45	20	60	35	10	50	25
1	26	1	41	16	56	31	6	46	21	61	36	11	51
2	52	27	2	42	17	57	32	7	47	22	62	37	12
3	13	53	28	3	43	18	58	33	8	48	23	63	38
4	39	14	54	29	4	44	19	59	34	9	49	24	64